Plant Disease Classification Using Classical Machine Learning and a Weighted Ensemble

***Abstract*—Plant disease may cause crop damage, reduce the quality of agricultural products, and result in economic losses due to symptoms not being detected on time. Diseases in plants are usually identified through observation of leaf samples; however, such identification methods are usually time-consuming and dependent on the inspector's experience. This paper describes a leaf-plant classification pipeline using classical machine learning algorithms and a weighted ensemble. It includes the use of images collected from PlantVillage and PlantDoc datasets, resizing, colour correction, contrast normalization, and leaf segmentation as pre-processing steps followed by representation of the images using colour descriptors, GLCM, LBP, and shape descriptors. K-Nearest Neighbours, SVM, and Random Forest classification algorithms were used with an aggregation of their predictions using weighted soft voting. A Flask-based web application allows the user to upload a sample of a leaf image and see the prediction results. According to the project documentation, 89.9% accuracy was achieved on the combined test set and 45.1% on PlantDoc field images. The gap between the two results highlights the difficulties of transferring model predictions from lab images to real-life photos.**

***Index Terms*—Plant Disease Detection, Traditional Machine Learning, Image Processing, GLCM, LBP, Support Vector Machine, Random Forest, Weighted Ensemble.**

I. Introduction

Since agriculture is important for livelihood and food production, losses resulting from disease infection can have effects that extend far beyond one particular piece of land. Disease infections in the form of damaged leaves, stems, or other parts of a plant affect the yield and the quality of the produce. Identifying the symptoms of disease helps farmers understand the issue and take appropriate preventive or remedial measures.

When it comes to practical application, disease diagnosis usually begins with symptoms visible to the eye: spots, discoloration, lesions, or a change in the shape of a leaf. Most experienced growers know how to identify such symptoms, yet the result may differ depending on the observer. Manual checking of many plants is rather time-consuming, especially when it comes to difficult-to-see symptoms or various diseases that are hard to differentiate.

The use of image processing allows identifying visual symptoms with consistency, whereas machine learning helps classify images into certain categories based on these symptoms [7]. Leaf color, texture, and shape give different information. Their numerical representation helps classify the picture comparing it to those used in training.

Convolutional neural networks and deep learning have been successful in image classification. On the other hand, such an approach may be computationally demanding and require a large number of training samples [3], [4], [11]. There is also a classic approach where features are extracted by image processing and conventional classifiers learn from these features [1], [7].

This paper considers a system which includes KNN, SVM, and Random Forest classifiers in a single pipeline where the results are fused using weighted soft voting. Image classification process, along with classification, also includes image pre-processing, feature extraction, and a web application. This means that the research task will focus on the description of leaf images in terms of color, texture, and shape features, the comparison of three classifiers, fusion of probability values, and making prediction possible via uploading an image.

II. Literature Review

The use of PlantVillage images has been shown by Mohanty et al. that deep learning models can be used in plant disease recognition [11]. The paper paved the way for image-based disease diagnosis as an area of interest. However, good performance on controlled photographs does not imply that the same would be obtained in outdoor images where the backgrounds and lighting may differ greatly.

PlantDoc has been proposed as an example of plant-disease images outside controlled condition [2], [8]. In such photos, illumination, background complexity, focus and distances may vary significantly. That is the reason why evaluation should incorporate both field-oriented and dataset images.

Haralick et al. have suggested several statistical texture measures based on the Gray-Level Co-occurrence Matrix (GLCM) [1], [6], [10]. The counts of gray-level occurrences next to each other make GLCM more informative than just mean colors in an image. LBP has been proposed by Ojala et al. as a way to describe the relationship between a pixel and its neighbors [9], [12].

The nearest-neighbour classifier was investigated by Cover and Hart [13]. The SVM technique implemented in this work is based on the Support Vector Networks, which were invented by Cortes and Vapnik [6], [9]. Random Forest by Breiman implements an alternative approach – it aggregates several decision trees instead of making decisions based on a single tree [13], [14]. Each of these techniques has its advantages, which makes their combination interesting for the current task.

In the current study, the colour, texture and shape features are fed to KNN, SVM and Random Forest, where they are aggregated using the weighted soft voting. Instead of learning each visual representation by means of a deep neural network,

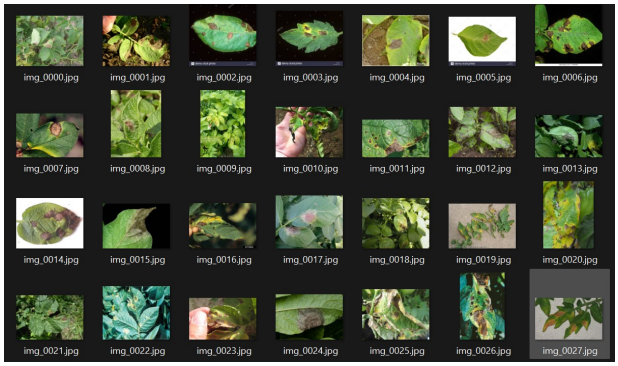


Fig. 1. Representative corn leaf images used for plant disease classification.

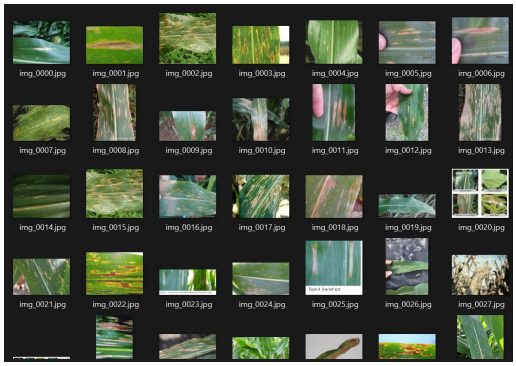


Fig. 2. Representative potato leaf images illustrating variations in disease symptoms.

the current pipeline uses the features which can be defined and thus inspected explicitly.

III. Dataset and Proposed Methodology

A. Dataset Description

The proposed system utilises plant leaf images from the PlantVillage and PlantDoc datasets. The images in the PlantVillage dataset are generally considered standard images used for classifying plant diseases, while images in the PlantDoc dataset are collected under more realistic conditions [2], [8]. The source manuscript includes a selected set of 770 images, including 560 images from the PlantVillage dataset and 210 images from the PlantDoc dataset, divided into seven classes. It also contains a more general overview of 5,447 images (5,000 images in PlantVillage and 447 images in PlantDoc) and 38 classes. These numbers correspond to different scopes of the dataset in the source paper and should not be considered as one identical split.

The project documentation further describes a final configuration of five disease classes and a 66-feature vector. Since the manuscript and the project report discuss two different experimental configurations, the final dataset manifest and feature extraction code should be referred to when determining the exact configuration for each experiment.

TABLE I
Dataset Summary

Parameter	Description
Dataset sources	PlantVillage and PlantDoc
Broader dataset total	5,447 images
PlantVillage images	5,000
PlantDoc images	447
Broader class count	38
Selected subset in text	770 images
Selected subset composition	560 PlantVillage; 210 PlantDoc
Classes in selected subset	7 (as stated in manuscript)

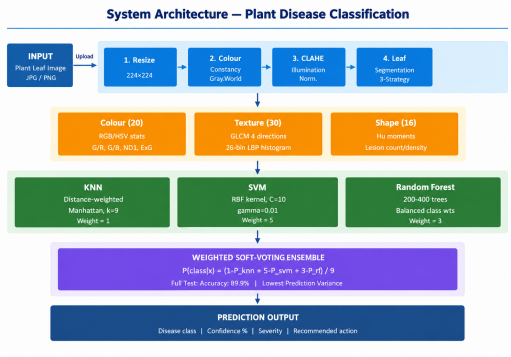


Fig. 3. System architecture of the plant disease classification pipeline.

B. System Architecture

The designed solution involves a series of processing steps. Leaf images are gathered and processed to enhance their visual appeal and remove unnecessary elements. Handcrafted features are extracted and fed into three different machine learning classifiers. The output of these classifiers is fused by using weighted soft voting to produce a final prediction which is then presented through the interface. The main stages are data gathering and organization, image preprocessing and leaf segmentation, feature extraction, training and hyperparameter tuning, weighted classification, performance evaluation, and prediction through the interface.

C. Image Preprocessing

Image preprocessing minimizes the impact of the variance between image quality, illumination, and background. Input images are uniformly scaled to a fixed size before additional processing; the exact size must match the implementation.

Colour constancy: The Gray-World assumption is utilized to mitigate the impact of illumination and make colors more consistent. Assuming $I_c(p)$ represents the intensity value of channel c at pixel p, the channel correction can be represented as

$$I'_c(p) = I_c(p)\frac{\mu}{\mu_c + \epsilon} \tag{1}$$

where μ_c is the mean intensity of channel c, μ is the mean across all channels, and ϵ guarantees that we do not divide by zero. This formula presents the general correction

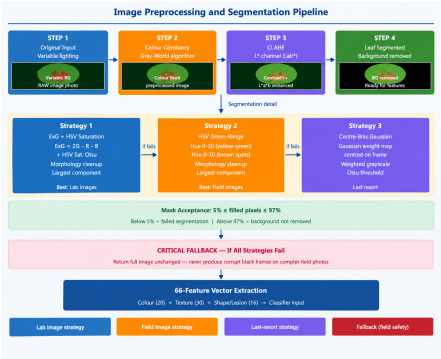


Fig. 4. Image preprocessing stages used before feature extraction.

technique; implementation details should match the project implementation.

Contrast enhancement: Contrast Limited Adaptive Histogram Equalization (CLAHE) is performed using the Lab color model. It increases local contrast without over-amplifying noise. The luminance channel is enhanced without altering chromatic channels.

Leaf Segmentation: The use of Excess Green index, HSV color information, morphological techniques, and connected components techniques help in the segmentation of the leaf from the background [9]. The equation for Excess Green index can be expressed as

$$\mathrm{ExG} = 2G - R - B. \tag{2}$$

Where R, G, and B are the values of red, green, and blue channels, respectively. A binary image may further be processed by opening and closing to eliminate the effect of noise and gaps.

D. Feature Extraction

The feature extraction phase converts visual attributes of the leaf images into values that the classifier can then analyze. There are three main categories of values generated by the pipeline: colour, texture, and shape features. Each category represents a unique aspect of the image; therefore, the overall feature vector may represent multiple symptoms.

Colour features: The colour features represent a distribution of pixel values in an image. Discoloration, spots, etc. affect this distribution in some way. As per the documentation provided for this project, there are 20 descriptors in the feature vector. Two basic statistical descriptors include mean and standard deviation:

$$\mu_c = \frac{1}{N}\sum_{p=1}^{N} I_c(p), \quad \sigma_c = \sqrt{\frac{1}{N}\sum_{p=1}^{N}(I_c(p) - \mu_c)^2}. \tag{3}$$

These descriptors describe the colour

distribution although they alone do not cover all lesion distributions.

Texture features: Texture features refer to local patterns and spatial relations of pixels. In this study, we employ GLCM and LBP [1], [12]. For a normalised co-occurrence matrix $P(i,j)$, some common GLCM characteristics are:

$$\text{Contrast} = \sum_{i,j}(i-j)^2 P(i,j), \tag{4}$$

$$\text{Energy} = \sum_{i,j} P(i,j)^2, \tag{5}$$

$$\text{Homogeneity} = \sum_{i,j} \frac{P(i,j)}{1+|i-j|}. \tag{6}$$

GLCM refers to the spatial relationships between the grey-level

intensities at certain locations. Correlation indicates the linear dependency between the grey levels and can be optionally used if supported by the implementation.

LBP refers to a measure of intensity patterns within a local neighbourhood using comparisons of the central pixel with its neighbours. The basic operator for LBP is

$$\mathrm{LBP}_{P,R}(x_c, y_c) = \sum_{p=0}^{P-1} s(g_p - g_c)2^p, \quad s(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases} \tag{7}$$

where g_c is the centre intensity and g_p is the intensity of

neighbour p. The histogram of the LBP code is a compact descriptor of the local texture. According to the project documentation, there are 30 texture descriptors. Shape features: Shape features characterise the contour and the geometry of the segmented leaf. Contour-based measurements may reflect differences between contours or shapes. An example of a frequently utilised metric is compactness:

$$C = \frac{4\pi A}{P^2 + \epsilon}, \tag{8}$$

where A is the area of the segmented region and P is its perimeter. The manuscript describes six shape features. The 66 features representation described in the project documentation comprises 20 colour, 30 texture, and 16 shape/pathology features, which contrasts with the 56-feature representation (20 colour, 30 texture, and six shape features) mentioned in the manuscript. The actual length of the vector should be determined based on the implementation of the feature extraction.

E. Why the Feature Groups Are Combined

The feature representation is created to capture several forms of evidences from the image of a leaf rather than focusing on only one visual hint. Colour-based descriptors will be able to indicate changes occurring due to yellowing, browning, as well as changes in the number of green pixels. Nevertheless, using colour may not always work correctly in case the image is taken in warm lighting, in shadows, or against a similar background. The process of pre-processing will help minimize

these differences, but it will be impossible to eliminate all differences.

Texture descriptors include additional information about the appearance of the leaf surface in the local region. GLCM feature extraction is based on the counting frequency of the grey-level co-occurrence pairs within certain displacements, enabling one to extract properties like contrast and homogeneity. The LBP operator characterises the local intensity transitions in relation to each point and allows describing the small patterns not considered by the global statistical characteristics of the colour histogram. These features are particularly valuable for the detection of the symptoms appearing in the form of spots or mottling as well as irregular surface of lesions. Nevertheless, their success is also related to the image resolution and the size of the lesion.

Shape descriptors offer an alternative kind of information by characterizing the boundary or the geometry of the region of interest. In this research, shape information is expected to complement color and texture information rather than serve as the only means of detecting disease. Most diseases do not affect the boundary shape significantly, hence shape information may be irrelevant for some categories. The accuracy of the segmentation is also crucial because inaccurate segmentation could result in the boundary information describing the segmentation errors instead of the plant itself.

This combination of feature sets will provide the classifiers with different types of information, but it will also make the preprocessing and the ordering of features even more critical. The pipeline for the training process and for the prediction on the website should compute the same features in the same order. Any feature normalization used during the training process needs to be used again during inference. The parameters used in the process should be saved along with the trained model and not be computed separately for every new image uploaded.

This means that the evaluation procedure needs to evaluate the whole pipeline and not just the classifier alone. It must verify whether the uploaded image gets decoded properly and resized based on the provided dimensions, then segmented in the same manner, and finally converted into a feature vector of the required size. Blurry images, images with strong shadows, images with overlapping leaves, or images with very few leaf parts visible need to be included as well in robustness testing since they may affect several components of the system simultaneously.

IV. Machine Learning Classifiers

A. K-Nearest Neighbours

K-Nearest Neighbours (KNN) makes predictions about a class on the basis of nearest training examples to the input feature vector [13]. The distance measure helps in determining which training examples are neighbours and hence feature scaling may have a significant impact on the output prediction. With feature vector $\mathbf{x}$ and training vector $\mathbf{x}_i$ consisting of d features, the Euclidean distance is given by

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}. \tag{9}$$

The KNN algorithm picks k number of nearest training examples and

classifies a class depending upon their class labels.

B. Support Vector Machine

Support Vector Machine (SVM) finds a decision boundary in the feature space to discriminate among different classes. SVM utilizes the radial basis function (RBF) kernel that makes it possible to model a nonlinear relationship between features [6]. For the case of two-class problem, the maximum margin optimization problem can be stated as

$$\min_{\mathbf{w},b,\{\xi_i\}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \tag{10}$$

subject to

$y_i(\mathbf{w}^\top \phi(\mathbf{x}_i)+b) \geq 1-\xi_i$ and $\xi_i \geq 0$. Here, C is a parameter that penalizes classification error. RBF kernel is defined as

$$K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma\|\mathbf{x} - \mathbf{z}\|^2). \tag{11}$$

where γ is a parameter that determines the influence of the spatial distribution of training samples. RBF kernel models nonlinear relationships between features.

C. Random Forest

Random Forest is an ensemble of different decision trees which classify data based on the majority of their votes [13], [14]. Each tree is created from randomly sampled data and using a different set of features; therefore, they all have slightly different properties. This helps avoid dependency on the peculiarities of one particular tree. In the case of a classification problem, a basic expression for the vote of a forest is

$$\hat{y} = \text{mode}\{T_1(\mathbf{x}), T_2(\mathbf{x}), \ldots, T_B(\mathbf{x})\}. \tag{12}$$

where T_b represents a prediction made by tree b, and B is

the number of trees. In case of probabilistic predictions, the class probability can be averaged over all trees. Random Forest can represent nonlinear dependencies and interaction between color, texture, and shape features. Its efficiency is parameterized by, for example, the number of trees, tree depth, and the number of features tested on each split.

D. Role of Each Classifier in the Pipeline

There are different perspectives on how the three classifiers arrive at the representation of features, which explains the need for joint assessment. The nearest neighbour classifier uses the proximity of the test sample to some training examples to come up with a decision. It is intuitive but depends highly on the feature scaling and the choice of k. In case the numerical range of colour measurements is substantially larger than that of texture measurements,

The distance calculations could be more affected by the colours of the features if they are not scaled in a similar way. KNN keeps the training data for the prediction process, which means that the inference time will increase as the reference dataset increases in size.

The SVM creates the separating boundary based on the training data. By using the RBF kernel, we can obtain a separating boundary that is non-linear without explicitly specifying a linear boundary in the original feature space. The value of the parameter C affects the balance between fitting the training data and making some classification errors, while the value of γ regulates how much impact individual samples have on the kernel. Extremes of both could cause bad generalization of the model, so those values should be chosen using the validation set rather than the test one.

Random Forest generates a number of decision trees, whose class predictions are then aggregated. Since different trees employ randomised instances and attributes, the forest is capable of modelling interactions between colour, texture and shape descriptors. It is not as dependent on the global distance measure like KNN, being able to model nonlinear relationships between features, yet it remains subject to parameters such as the number of trees, their depth and the minimum size of a leaf node. The comparative power of these techniques could differ depending on the particular class, which can be dominated by texture or colour changes.

These are some of the reasons that suggest that it is worth studying the idea of using a weighted ensemble of the classifiers, but this does not mean that the fusion of these classifiers would necessarily lead to better results. In case the models have the same types of mistakes, the voting would provide little value, whereas in case the types of the mistakes made by different models differ, the fusion would help in making more reliable decisions. That is why the report must provide precision, recall and F1-score of each model for each class as well as the result of the ensemble.

V. Weighted Soft-Voting Ensemble

The ensemble combines the methods of KNN, SVM, and Random Forest using weighted soft voting [10], [13]. Unlike hard voting, which considers the winners of each classification model, soft voting takes into account the probabilities associated with all possible classes. In this way, each model is allowed to have a different influence on the ultimate decision. Here, let $P_i(c \mid \mathbf{x})$ denote the probability associated with class c produced by classifier i on image $\mathbf{x}$ and w_i denotes its weight. Then, the combined probability will be

$$P_{\text{ens}}(c \mid \mathbf{x}) = \frac{\sum_{i=1}^{m} w_i P_i(c \mid \mathbf{x})}{\sum_{i=1}^{m} w_i}. \tag{13}$$

where m is the total number of classifiers, and $w_i \geq 0$. The final class prediction will be

$$\hat{c} = \arg\max_{c \in \mathcal{C}} P_{\text{ens}}(c \mid \mathbf{x}). \tag{14}$$

In the current approach, classifier weights depend on the validation accuracy. Thus, classifiers with higher validation accuracy get more weight in the combined prediction. Weighted voting allows us to use several models instead of just one classifier. Nonetheless, it is required to conduct an experimental evaluation of the ensemble to understand whether the latter works better than the classifiers themselves.

TABLE II
Technology stack used in the proposed system

Component	Technology or method
Programming language	Python
Image processing	OpenCV
Numerical computation	NumPy
Data handling	pandas
Machine learning	scikit-learn
Feature extraction	GLCM, LBP, colour and contour descriptors
Hyperparameter tuning	GridSearchCV
Web framework	Flask
Datasets	PlantVillage and PlantDoc

VI. Model Training and Hyperparameter Tuning

The experiment used in the documentation is a stratified 70:15:15 ratio for training, validation, and testing. Fitting of the model happens in the training data set, the validation data set helps to determine the optimal values of the hyperparameters and the ensemble weights, and the test data set will be used at the very end for evaluation. Stratification seeks to make sure that the proportion of the classes in each of the data sets is similar, but the numbers used should also be captured.

GridSearchCV is used in the experiment to help compare various hyperparameter configurations. The experiment used in the documentation also uses the augmentation of training images in order to expose the models to possible variations [5]. This can be through rotation, flipping, cropping or changes in illumination depending on the image and its corresponding label. This should happen after the dataset has been split up and only the training images can undergo the process.

Samples from PlantDoc get additional weights in the training procedure, as specified in the project description. The specifics of the procedure and the reasoning behind it must be described according to the code. This weight is used for increasing the impact of field images on the fitting, although it does not substitute for good field image coverage. Reporting reproducibly requires the split seed, class counts, preprocessing parameters, features, and chosen hyperparameters.

VII. Domain-Aware Evaluation and Reproducibility

As the image sources depict various imaging conditions, the source of each image must be preserved while evaluating it. If the purpose of the test is the classification of images taken in farm conditions, then a random partitioning where nearly identical imaging conditions are found in both the training and testing sets would result in an overly optimistic performance assessment. In addition to the stratified partitioning, a source-separate experiment can also be conducted using PlantVillage images for training and PlantDoc images for testing.

In order to have a fair comparison, the same held out test data will need to be used when performing KNN, SVM, Random Forest and weighted ensembling techniques. The validation data will be used to determine the hyperparameters and weights for the classifiers while the test data will not be touched until the results are finalized. The preprocessing parameters and feature scaling will need to be determined based on learning from the data that does not use test data labels. For images which come from the same plant or imaging session, it is ideal for them to be in the same partition.

The reproducible experiment would include information about the number of images per class, labeling of data sources, the seed used for splitting, the resized image dimensions, the segmentation parameters, the ordering of features, the scaling process, the hyper-parameters of the classifier used, and the ensemble weights. The vector of features that is saved during training should have the same size and ordering as the one generated after uploading an image by the user. This is especially important in this case since the manuscript refers to 56 features but the project documentation refers to 66 features.

The augmentation process must take into account realistic transformations of images captured in the field. These include small transformations like rotations, horizontal flips where applicable, minor adjustments to brightness levels, and cropping when necessary, but they must not result in the loss of the observable lesion or such an alteration of the image that would make it difficult to label the image. The artificial versions of the image must be created after dividing the data set and using only the training images.

VIII. Implementation

The suggested approach has been developed on the basis of using Python language together with Flask library and some popular machine learning and image processing packages. In the process of work, Flask performs HTTP request processing, handling of image uploading and sending results of classification through the web interface. Implementation includes modules for data preparation, image preprocessing, feature extraction, model training, validation, and prediction.

IX. User Interface

The Flask-based user interface will make the system of classification easy to use by allowing users to interact with the system through an interactive interface that will eliminate the need to manually run the training process. The user interface allows the user to upload an image of the plant leaf to be tested. The uploaded image undergoes the same processes of preprocessing and feature extraction as were done during the training process.

The user interface can be extended to provide additional information including confidence scores, prediction results for individual classifiers, and information about the disease that has been predicted. Interpretation of the predictions should not be seen as replacing the expertise of agricultural professionals in case there are notable differences between the image and the training data.

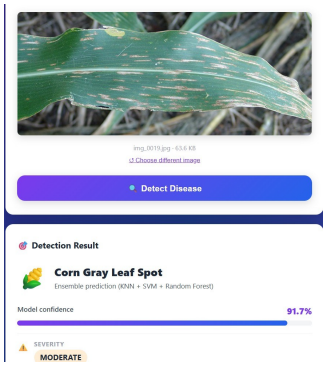


Fig. 5. Plant disease detection result for Corn Gray Leaf Spot, including model confidence and severity.

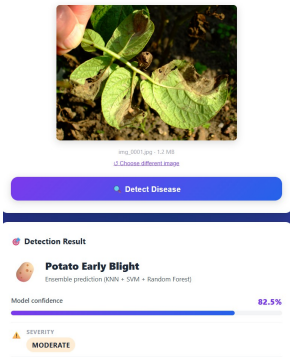


Fig. 6. Prediction output for an uploaded potato leaf image, showing Potato Early Blight with 82.5% model confidence and moderate severity.

X. Experimental Evaluation and Discussion

The test set is the most reliable source to report performance results. Accuracy is a good measure, but can give an inaccurate performance for uncommon classes when there are imbalanced class frequencies. Therefore, the two classifiers must be evaluated using the same test set.

A. Evaluation Metrics

Accuracy is the ratio of correctly classified images:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{15}$$

Precision for a certain class is the ratio of true positives among predicted positive cases:

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{16}$$

Recall is the ratio of true positives among all positive cases:

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{17}$$

Harmonic mean of precision and recall is called F1 score:

$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{18}$$

Where TP, TN, FP and FN stand for true positives, true negatives, false positives and false negatives correspondingly. In the multi-class classification case macro averaged precision, recall and F1 score are helpful as they give each class equal weight. For K classes macro-F1 is

$$F_{1,\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} F_{1,k}. \tag{19}$$

Confusion matrix should also be produced to detect classes which are often confused with other ones.

B. Results and Interpretation

From the project report, the algorithm attained an accuracy rate of 89.9% on the combined test set and 45.1% on the PlantDoc real-world image dataset. There is obviously a considerable difference between these numbers and it implies that the system performance on a mixed or controlled dataset cannot be assumed to generalize to field images. Field photographs may have different elements such as occlusion, presence of multiple leaves, complicated background, variations in distance to the camera, and unclear symptoms of plant diseases as compared to photographs taken under more controlled conditions with possibly clearer backgrounds and more uniform lighting.

The lower value on PlantDoc data is critical for evaluating practical utility of the algorithm. This means that future work should concentrate on data variation and generalization beyond a large aggregate accuracy rate. From the project report, there are five classes and a 66-dimensional feature vector whereas from the manuscript draft, there are seven selected classes and 56 features. The accuracy values presented here should thus be cited alongside the particular dataset split and feature vector used; the currently available documents do not provide evidence that all these values are from the same experiment.

These measures should be combined with the visualization output and confusion matrix. It is possible that two disease classes have very similar colour change or lesions and even if the overall accuracy is good, it may still misclassify those two classes. Class-wise recall can help detect which disease class is being often overlooked by the classifier, while class-wise precision may help detect which classes are being overpredicted. Although macro-F1 measure is helpful when the number of classes is unequal, it cannot substitute for looking at class-wise measures. The classification report from Fig. [7] should therefore be examined together with the exact split and class mapping.

The combined score and the PlantDoc score address two separate issues. The combined score shows how well the algorithm works on the reported mixture of the test dataset, whereas the PlantDoc score shows the problem of working with field-oriented photos during the documented experiment. Neither measure can independently confirm the robustness of the algorithm for every possible crop, disease state, camera, or farming area.

Classification Report — ensemble_full

	precision	recall	f1-score	support
Corn_Gray_Leaf_Spot	0.96	0.97	0.97	160
Potato_Early_Blight	0.89	0.90	0.90	168
Tomato_Early_Blight	0.84	0.89	0.87	164
Tomato_Healthy	0.91	0.97	0.94	159
Tomato_Late_Blight	0.88	0.76	0.82	167
accuracy			0.90	818
macro avg	0.90	0.90	0.90	818
weighted avg	0.90	0.90	0.90	818

Accuracy : 0.8985
Precision : 0.8988 (macro)
Recall : 0.8999 (macro)
F1-score : 0.8981 (macro)

Fig. 7. Classification report of the weighted ensemble, as shown in the supplied manuscript.

XI. Limitations

The accuracy of the proposed approach relies on the quality and variety of images used. The image set of small size may not include all types of plants, diseases, lighting conditions, and environments.

The use of hand-crafted features may not be able to account for all complex visual patterns that are related to plant diseases. Segmentation errors might influence the accuracy of the features extraction process too. In addition, the ensemble model does not necessarily perform better than each individual classifier; this fact needs to be confirmed experimentally using the same test set. Differences in the number of images and features vectors description in the manuscript and project documentation need to be clarified first.

The prediction is based on an image and must not be considered as a substitute for professional agricultural diagnosis before validation in practical applications. First of all, confidence scores are model outputs and cannot be treated as calibrated probability scores until such validation is provided. There are also some practical issues that remain unaddressed by the current assessment of the model: does the model work consistently for different cameras, are symptoms visible in the early stages distinct from normal variation, and will the same level of performance be maintained if several leaves or other objects are present in a single image?

To ensure the reproducibility of the experiments, it is important to have information about the dataset version, classes used, preprocessing techniques, feature engineering details, classifier parameters, voting weights, and train-test split for every result presented. The same set of held-out instances must be used in order to compare KNN, SVM, Random Forest, and the ensemble. It would make it possible to know if the improvement is due to feature engineering, classifier used, or the voting technique.

XII. Conclusion and Future Work

This study presents a disease classification system based on classical machine learning and weighted soft voting. After pre-processing and segmentation of the images, color, texture, and shape features are extracted for use by KNN, SVM, and

Random Forest. The corresponding Flask application provides an easy way for users to upload a leaf picture and see the prediction of the model.

Weighted soft voting is the method of forming one prediction using the probability scores returned by individual classifiers. Since the features are explicitly defined by this pipeline, this method allows one to get a more interpretable system compared to pure deep learning. As reported in the project documentation, this project achieved 89.9% accuracy on the unified test set and 45.1% on PlantDoc field images.

The prediction itself will have to be seen as a preliminary screening result. One can compare the classification given by the program with symptoms one observes and double-check the prediction when the image is unclear or the result does not correspond to observations made. The point is that the system should not provide any prescription based on the prediction made, since the current experiments do not allow to prove the system's performance in all cases of crops, regions, and stages of development of diseases.

Next steps include finalizing and documenting the experiment itself, optimizing parameters of feature extraction and classification, running the experiment on other real images, and developing interface with confidence measures and description of particular diseases. In addition, the interface can give the user brief description of the predicted class and ask to take another picture when the leaf on the picture is blurred, insufficiently lighted, or partially covered.

The additional work should include enlarging the PlantDoc dataset, applying source separation for evaluation, presenting per-class results and confusion matrices, and confirming the number of features and dataset configuration according to the actual code. The experiments should involve comparison of every base classifier against the ensemble in question using the same splitting and explaining the way of choosing voting weights. Further validation by agronomists is required before making decisions based on the results of the system. For the long-term perspective, it is necessary to collect images from different crops, regions, devices, and stages of diseases.

REFERENCES

[1] Urfiyati et al., "Classification of Banana Leaf and Ornamental Plant Diseases Using GLCM and Hybrid Random Forest–SVM," Jurnal Teknik Informatika (JUTIF), 2026.

[2] Salman et al., "Plant Disease Classification in the Wild Using Vision Transformers and Mixture of Experts," Frontiers in Plant Science, vol. 16, Art. no. 1522985, 2025.

[3] Jiang et al., "PlantCaFo: An Efficient Few-Shot Plant Disease Recognition Method Based on Foundation Models," Plant Phenomics, vol. 7, Art. no. 100024, 2025.

[4] S. Murugavalli and R. Gopi, "Plant Leaf Disease Detection Using Vision Transformers for Precision Agriculture," Scientific Reports, 2025.

[5] Antwi et al., "On the Application of Image Augmentation for Plant Disease Detection: A Systematic Literature Review," Smart Agricultural Technology, vol. 9, Art. no. 100590, 2024.

[6] Khan et al., "Automated Tomato Leaf Disease Detection Using Image Processing: An SVM-Based Approach with GLCM and SIFT Features," Journal of Engineering, 2024.

[7] Demilie, "Plant Disease Detection and Classification Techniques: A Comparative Study of the Performances," Journal of Big Data, vol. 11, Art. no. 5, 2024.

[8] Wu et al., "From Laboratory to Field: Unsupervised Domain Adaptation for Plant Disease Recognition in the Wild," Plant Phenomics, 2023.

[9] Jamjoom et al., "Plant Leaf Diseases Classification Using Improved K-Means Clustering and SVM Algorithm for Segmentation," Computers, Materials \{}& Continua, 2023.

[10] Mathew et al., "Plant Disease Detection Using GLCM Feature Extractor and Voting Classification Approach," Materials Today: Proceedings, 2022.

[11] R. Sharma et al., "Plant Disease Diagnosis and Image Classification Using Deep Learning," Computers, Materials \{}& Continua, 2022.

[12] K. Trang, L. TonThat, K.-L. Nguyen, and G.-H. Tran, "Identification of Plant Disease Based on Multi-feature Extraction," Springer, 2022, doi: 10.1007/978-3-030-75506-5_44.

[13] A. Yousuf and U. Khan, "Ensemble Classifier for Plant Disease Detection," Int. J. Comput. Sci. Mobile Comput., 2021.

[14] A. Tabbakh and S. S. Barpanda, "Evaluation of Machine Learning Models for Plant Disease Classification Using Modified GLCM and Wavelet Based Statistical Features," Traitement du Signal, vol. 39, no. 6, pp. 1893–1905, Dec. 2022, doi: 10.18280/ts.390602.